

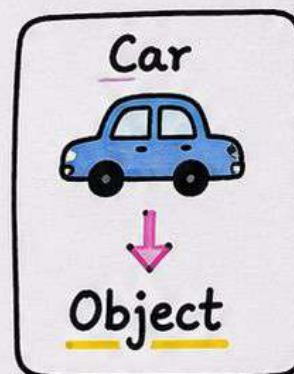


OOPs (Object-Oriented Programming)

Introduction to OOPs

★ What is OOPs?

OOPs stands for Object-Oriented Programming. It is a programming paradigm based on the concept of "Objects", which can contain data (attributes) and code (methods).



★ Why OOPs was introduced?

- Procedural programming was becoming complex for large projects.
- Hard to maintain, modify and reuse code.
- OOPs makes code more scalable, modular, and easy to maintain.



★ Procedural vs Object-Oriented Programming

Procedural Programming	Object-Oriented Programming
→ Focus on functions. → Program is divided into functions. → Data is separate from functions. → Less secure. → Difficult to maintain large code.	→ Focus on objects. → Program is divided into objects. → Data and functions are bundled together. → More secure. → Easy to maintain and reuse.

★ Real-life Examples

- A Car has attributes (color, speed) and behaviors (start(), stop()).
- A Student has attributes (name, age) and behaviors (study(), attend()).



Think of real-world entities as objects!

★ Languages supporting OOPs

- Java
- C++
- Python
- C#
- JavaScript
- PHP

Almost all modern languages support OOPs!

OOPs makes code more organized, reusable and closer to real life. ❤️



Class & Object

★ What is a Class?

A class is a **blueprint** or template used to create objects. It defines a set of **attributes** (data) and **methods** (functions) that the objects created from the class will have.

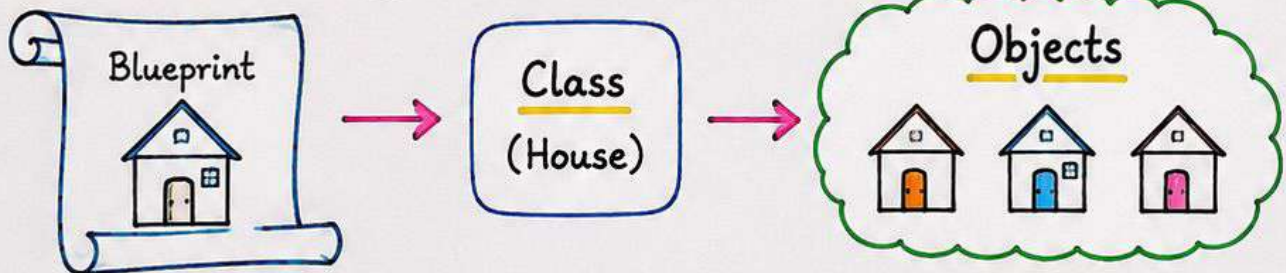
★ What is an Object?

An object is an **instance** of a class. It is a real-world entity that has **state** (data) and **behavior** (methods).

Class is the **plan**,
Object is the **real**
thing!



★ Real-world Analogy.



Blueprint → Class → Many Objects

★ Syntax (Example)

```
class Car {  
    String color;  
    int speed;  
    void start() {  
        System.out.println("Car Started");  
    }  
}
```



Class is just a template.
Objects are created to use it!

★ Creating Objects (Example)

```
Car car1 = new Car();    // object 1  
Car car2 = new Car();    // object 2
```



Key Point

- Class is user-defined data type.
- Objects occupy memory.
- Each object has its own copy of data (attributes).



➔ First create Class, then Objects, and then call Methods to perform actions! 😊



Four Pillars of OOPs

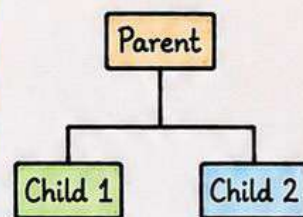
OOPs is built on 4 main principles, also called the Four Pillars of OOPs.

1. Encapsulation



Binding data (variables) and methods together in a single unit (class) and restricting direct access to some of the object's components.

2. Inheritance



Mechanism where a new class (child) acquires properties and behaviors of an existing class (parent).

3. Polymorphism



Same sound
"Bark"

Ability of an object to take many forms. The same method name can behave differently based on the object.

4. Abstraction



I just drive,
don't know
how it works
inside!

Hiding internal implementation details and showing only the essential features to the user.

Why these pillars?

- ★ They make the program more modular.
- ★ Code becomes reusable and easy to maintain.
- ★ Helps in building scalable and secure applications.
- ★ Reduces complexity and promotes real-world modeling.

Remember:

Encapsulation binds,
Inheritance extends,
Polymorphism adapts,
Abstraction simplifies.



Together these 4 pillars make OOPs powerful, flexible and closer to how we solve real-life problems! 😊



ENCAPSULATION

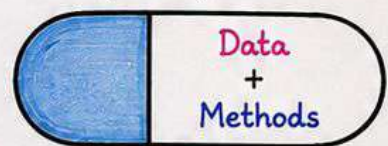
Encapsulation is the mechanism of binding data (variables) and methods together and restricting direct access to some of an object's components.

★ Key Points

- Wraps data and functions into a single unit (class).
- Hides the internal details and shows only necessary features.
- Achieved using **private** variables and public **getter/setter** methods.
- Helps in **data** protection and controlled access.
- Improves flexibility and maintainability.

Encapsulation Analogy

Think of a capsule.



Internal details are hidden inside the capsule.

Only necessary things are exposed to the user.

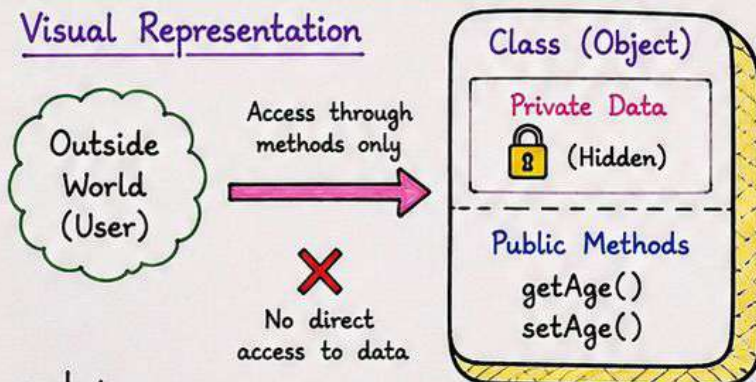
★ Example (Java)

```
class Student {
    // private data (hidden)
    private int age;
    // public methods to access & modify data
    public int getAge() {
        return age;
    }
    public void setAge(int a) {
        age = a;
    }
}
```

How it works?

- ① Private variable 'age' cannot be accessed directly outside the class.
- ② We use public methods **getAge()** and **setAge()** to read and modify it.
- ③ This protects data from unwanted modification.
- ④ We have full control over how data is accessed and updated.

Visual Representation



Advantages

- ✓ Protects data from being accessed by unauthorized code.
- ✓ Helps in making the program more secure.
- ✓ Reduces impact of changes in code.
- ✓ Makes code easier to maintain and update.



Encapsulation is all about: "Hide the data, show the behavior."
It ensures safety, simplicity and control. 😊



ABSTRACTION

Abstraction is the process of **hiding** complex implementation details and showing only the **essential** features to the user.

★ Key Points

- Focuses on **WHAT** an object does, not **HOW** it does.
- Hides unnecessary details and shows only **important** information.
- Achieved using **Abstract Classes** and **Interfaces**.
- Helps in reducing complexity and increasing security.
- Improves code **maintainability** and **reusability**.

Abstract Class (Java)

- A class that **cannot** be instantiated.
- Can contain both **abstract** methods (no body) and **concrete** methods (with body).
- Used when we want to provide a common base with some shared behavior.

```
abstract class Shape {
    abstract void draw(); // no body
    void color() {
        System.out.println("Coloring shape");
    }
}
```

Subclasses
must provide
body for
draw().

Why Abstraction?

- ✓ Hides **complexity** from the user.
- ✓ Helps in building **scalable** systems.
- ✓ Makes code **easier to understand**.
- ✓ Improves **security** by hiding sensitive details.

Real-life Analogy



You drive a car
(Use the features)



Engine, gears, wires,
mechanisms, etc.



You don't know the
internal working details.

You only use what is **essential**!

Interface (Java)

- A **fully abstract** type.
- All methods are **public** and **abstract** by default. (No method body)
- Variables are **public**, **static** and **final** by default.
- Used to achieve **multiple inheritance** in Java.

```
interface Drawable {
    void draw(); // public abstract
    void resize(); // public abstract
}
```

No body
in interface
methods

Remember

Think of Abstraction as
"Show the **essential**,
Hide the rest."



★ Abstraction is all about focusing on **WHAT** matters, not **HOW** it works! 😊



INHERITANCE

Inheritance is a mechanism in OOPs where a new class acquires the properties and behaviors (methods) of an existing class. The existing class is called **Parent** (or Base) and the new class is called **Child** (or Derived).

★ Key Points

- Promotes **code reusability**.
- Establishes a relationship between parent and **child** class.
- Child class can **add new features** or **override** existing ones.
- Provides a natural way of organizing and structuring code.
- Follows "**IS-A**" relationship.

Basic Syntax (Java)

```
class Parent {
    // fields and methods
}
```

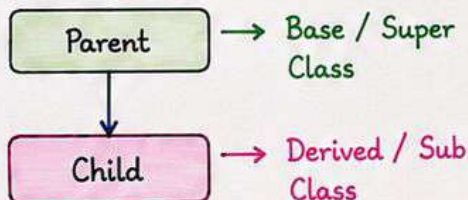
→ Parent Class

```
class Child extends Parent {
    // new fields and methods
}
```

→ Child Class

→ inherits from Parent

Class Diagram



Real-life Analogy

Animal



(Parent)

IS-A →

Dog

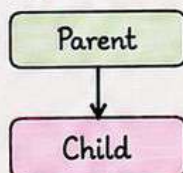


(Child)

Dog is an Animal. It inherits all the properties of Animal and can have its own behaviors. ❤️

Types of Inheritance

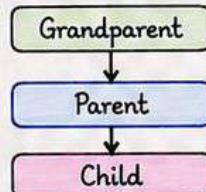
1. Single Inheritance



One child class inherits from one parent class.

Example: **Dog** inherits from **Animal**.

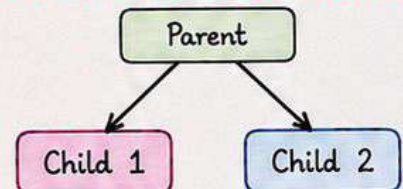
2. Multilevel Inheritance



A child inherits from a parent, which itself inherits from another class.

Example: **Baby** → **Person** → **Animal**.

3. Hierarchical Inheritance



Multiple child classes inherit from the same parent class.

Example: **Dog**, **Cat** inherit from **Animal**.



Inheritance helps us build software that is **reusable**, **extensible** and **easy to maintain**. 😊

Reuse is Smart!



POLYMORPHISM

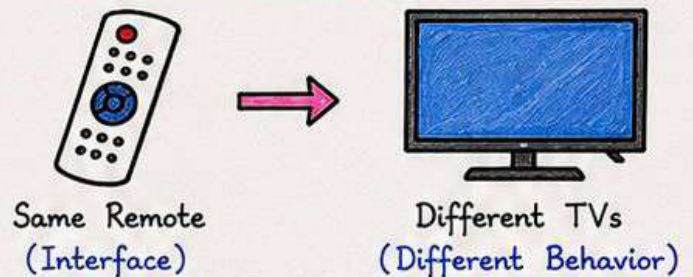
Polymorphism means "Many Forms".

It allows objects of different classes to be treated as objects of a common superclass.

★ Key Points

- One interface, multiple implementations.
- Same method name, different behavior.
- Increases flexibility and extensibility of code.
- Achieved using **Method Overloading** (compile-time) and **Method Overriding** (runtime).

Real-life Analogy



Pressing "Power" button works differently on different TVs.

Types of Polymorphism

1. Compile-time Polymorphism

(Static Polymorphism / Early Binding)

Method Overloading

Same method name with different parameters (number, type or order).
Decided at **compile-time**.

```
class Math {
    int add(int a, int b) { ... }
    double add(double a, double b) { ... }
    int add(int a, int b, int c) { ... }
}
```

Rules for Overloading

- Method name must be same.
- Parameter list must be different. (type / number / order)
- Return type can be same or different.
- Access modifier can be same or different.

2. Runtime Polymorphism

(Dynamic Polymorphism / Late Binding)

Method Overriding

Same method name and same parameters in parent and child class.
Decided at **runtime**.

```
class Animal {
    void sound() { System.out.println("Animal Sound"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Bark"); }
}
```

Rules for Overriding

- Method name must be same.
- Parameter list must be same.
- Return type must be same or covariant.
- Access modifier in child cannot be more restrictive than parent.

Quick Comparison



CONSTRUCTOR

A Constructor is a special method that is automatically called when an object is created to initialize the object.

Constructor name = Class name

★ Key Points

- Constructor has **no return type** (not even void).
- Constructor name must be **same as class name**.
- It is called **automatically** when object is created.
- Used to **initialize** objects.
- We **cannot** call constructor explicitly.
- Constructor can be **overloaded**.

Constructor Syntax (Java)

```
class ClassName {
    ClassName() {
        // constructor body
    }
}
```

Called automatically!



Types of Constructors

1. Default Constructor (No-arg Constructor)

- Takes **no parameters**.
- If we don't create any constructor, Java provides a **default constructor**.

```
class Student {
    int rollNo;
    String name;
    // default constructor
    Student() {
        rollNo = 0;
        name = "Unknown";
    }
}
```

Provided by Java (if you don't write any constructor)

2. Parameterized Constructor

- Takes **one or more parameters**.
- Used to **initialize values** at the time of object creation.

```
class Student {
    int rollNo;
    String name;
    // parameterized constructor
    Student(int r, String n) {
        rollNo = r;
        name = n;
    }
}
```

Example

Default Constructor

```
Student s1 = new Student();
System.out.println(s1.rollNo); // 0
System.out.println(s1.name); // Unknown
```

Parameterized Constructor

```
Student s2 = new Student(101, "Abhi");
System.out.println(s2.rollNo); // 101
System.out.println(s2.name); // Abhi
```

Constructor vs Method

Constructor	Method
No return type	Must have return type (or void)
Name same as class	Name can be anything
Called automatically	Called explicitly



THIS & SUPER KEYWORD

this and **super** are reference variables in Java. They are used to remove **ambiguity** and access members of the class and parent class.

1. this Keyword

- Refers to the **current object**.
- Used to access current class **instance variables**.
- Used to invoke current class **methods** (implicitly).
- Used to call another constructor of the **same class**.

Example

```
class Student {
    int rollNo;
    String name;
    Student(int rollNo, String name) {
        this.rollNo = rollNo; // uses this to
        this.name = name;     // refer current
    }                          object
    void display() {
        System.out.println(this.name);
    }
}
```

this is used to remove name conflict



2. super Keyword

- Refers to the **parent class object**.
- Used to access **parent class variables**.
- Used to access **parent class methods**.
- Used to call **parent class constructor**.

Example

```
class Animal {
    String type = "Animal";
    void show() {
        System.out.println("I am Animal");
    }
}
class Dog extends Animal {
    String type = "Dog";
    void show() {
        System.out.println("super.type"); // Animal
        super.show();                     // calls parent method
    }
    Dog() {
        super();                          // calls parent constructor
    }
}
```

super is used to access parent members



this vs super

Feature	this	super
Refers to	Current class object	Parent class object
Used to access	Current class members	Parent class members
Constructor call	this() → calls another constructor in the same class	super() → calls parent class constructor
Method call	this.method() → current class method	super.method() → parent class method
Variable access	this.variable → current class variable	super.variable → parent class variable
Usage	Used within the same class	Used within child class (inheritance)

Constructor Chaining

Using this() or super(), we can call one constructor from another.

- this() → must be the **first statement**

Rule:
this() or
super()

Real-life Analogy

Child
(You)

this → You
(talk about yourself)

Parent
(Your Father)



METHOD OVERRIDING

Method Overriding occurs when a child class provides a **specific implementation** of a method that is already defined in its parent class.

★ Key Points

- Occurs in **inheritance**.
- Same** method name, same parameters.
- Return type must be **same** or covariant.
- Access level in child cannot be **more restrictive**.
- Used to provide **specific behavior** in child class.

Example (Java)

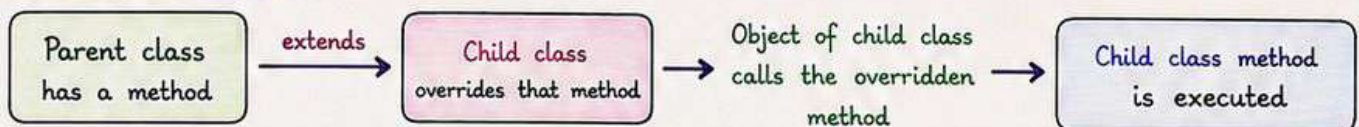
```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
```

Child class provides its own version of **sound()**



How It Works ?



Overriding Rules

Rule	Description	Example / Note
Same Method Signature	Method name and parameters must be same.	sound() in parent and child must be identical .
Return Type	Must be same type or covariant return type.	Animal type → Animal type Animal type → Dog type (✓)
Access Modifier	Child method cannot be more restrictive than parent.	public → public ✓ public → private ✗
Exception	Child method cannot throw broader checked exceptions.	If parent throws IOException , child cannot throw Exception .
@Override Annotation	It is optional but highly recommended.	Helps compiler validate overriding and avoid mistakes.

Example: Overriding in Action

```
Animal a = new Dog(); // upcasting
a.sound();             // Dog barks
```

Why Use Overriding ?

- To achieve **runtime polymorphism**.

Real-life Analogy

Parents (Animal) have a common action (sound). Children (Dog, Cat) perform their own version of sound.



ABSTRACTION

Abstraction means "showing only essential details" and hiding the implementation.

It reduces complexity and focuses on **what** an object does, not **how** it does.

★ Key Points

- Hides internal implementation and shows only functionality.
- Helps in **reducing** complexity.
- Improves **security** by hiding sensitive details.
- Achieved using **abstract classes** and **interfaces**.

Real-life Analogy



When we drive a car, we only use the **steering**, **brakes** and **accelerator**.



We don't need to know **how** the **engine** actually works inside.

We focus on what it does, not **how** it does it. 😊

Ways to Achieve Abstraction in Java

1. Abstract Class

- Can have **abstract methods** (no body) and **concrete methods** (with body).
- Cannot be instantiated.
- Child class must provide implementation for abstract methods.

```
abstract class Shape {
    abstract void draw(); // abstract method
}
class Circle extends Shape {
    void draw() {
        System.out.println("Drawing Circle");
    }
}
```

Shape
is an
abstract
class
😊

2. Interface

- Can have only **abstract methods** (by default **public** and **abstract**).
- Cannot be instantiated.
- Class implements interface using **implements** keyword.

```
interface Animal {
    void sound(); // abstract by default
}
class Dog implements Animal {
    public void sound() {
        System.out.println("Bark");
    }
}
```

Animal
is an
interface
😊

Abstract Class vs Interface

Feature	Abstract Class	Interface
Methods	Can have abstract + concrete methods	Only abstract methods (by default)
Variables	Can have instance variables	Only public static final variables

Remember

- ★ Abstraction is about "what", not "how".



ENCAPSULATION

Encapsulation means "wrapping data (variables) and code (methods) together into a single unit (class)" and restricting direct access to the data.

★ Key Points

- Data is **hidden** using access modifiers (private).
- Accessed only through public methods (getters & setters).
- Protects data from **invalid modification**.
- Improves **security** and **maintainability**.

Example (Java)

```
class Student {
    private int age;    // hidden data

    public int getAge() {    // getter
        return age;
    }

    public void setAge(int age) {    // setter
        if (age > 0) {
            this.age = age;
        }
    }
}
```

age is **private**,
so we cannot
access it directly
from outside!
😊

Without Encapsulation (Problem)

```
class Student {
    int age;    // public - directly accessible
}

Student s = new Student();
s.age = -10;    // ❌ Invalid data!
```



With Encapsulation (Solution)

```
Student s = new Student();
s.setAge(-10);    // ❌ Not allowed
s.setAge(22);    // ✅ Valid data
System.out.println(s.getAge());    // 22
```

Encapsulation vs Abstraction

Feature	Encapsulation	Abstraction
Focus	Data Hiding	Hiding Implementation
Achieved using	Access Modifiers	Abstract class / Interface
Goal	Protect data	Show only essential behavior
Level	Field level (variables)	Method level (functionality)



Benefits of Encapsulation

- ✓ Protects data from unwanted access.
- ✓ Ensures better control over data.
- ✓ Makes code more **flexible** and **maintainable**.
- ✓ Helps in achieving **read-only** or **write-only** properties.
- ✓ Provides **security**.

Real-life Analogy

Like a Capsule Medicine

How to Achieve ?

1. Declare variables as **private**.

Remember



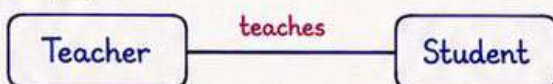
ASSOCIATION, AGGREGATION & COMPOSITION

These are different types of **relationships** that describe how objects are related to each other in OOP. 😊

1. Association

- A general relationship between two classes.
- Objects of one class are aware of the other class.
- It can be **one-to-one**, **one-to-many**, **many-to-one** or **many-to-many**.

Example :

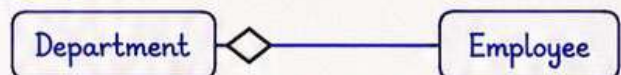


A teacher teaches students.
Both can exist independently. 😊

2. Aggregation

- A special form of association.
- Represents a **"has-a"** relationship.
- The child object can exist **independently** of the parent.
- Shown by a **hollow diamond (◇)** near the parent.

Example :



A department has employees.
Employees can exist without the department. 😊

3. Composition

- A strong form of aggregation.
- Represents a **"part-of"** relationship.
- The child object **cannot** exist without the parent.
- Shown by a **filled diamond (◆)** near the parent.

Example :



A car has an engine.
Engine **cannot** exist without the car. 😊

Real-life Analogy →

Association

A student studies in a university. They are related but can exist separately.

Aggregation

A house has windows. Windows can exist independently.

Composition

A car has an engine. Engine **cannot** exist without the car.

Comparison Table

Feature	Association	Aggregation	Composition
Meaning	General relationship	Special "has-a" relationship	Strong "part-of" relationship
Type	Weak	Medium	Strong



WHEN TO USE WHICH ?

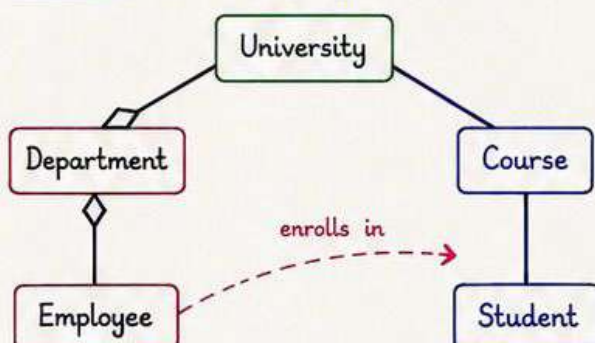
Choosing the **right relationship** helps in better design, code **reusability** and **maintainability**. 😊

Criteria	Association	Aggregation	Composition
Meaning	General relationship between two classes.	"Has-a" relationship. Child can exist independently.	"Part-of" relationship. Child cannot exist without parent.
Dependency	Objects are independent.	Weak ownership.	Strong ownership.
Lifetime	Both have their own lifecycle.	Child can live even if parent is destroyed.	Child dies when parent is destroyed.
Symbol	Simple line ———	Hollow diamond ———◇———	Filled diamond ———◆———
Use When	Two classes are related but no ownership.	Whole is made of parts but parts can exist separately.	Whole has parts and parts are dependent on whole.
Example	Doctor — Patient Student — Course	Team ◇ Player Department ◇ Employee	House ◆ Room Car ◆ Engine

Multiple Relationships Together

A class can have more than one type of relationship with different classes.

Example : University System



★ University has Departments

Common Examples in Real Life

- A library has books. (**Composition**)
- A team has players. (**Aggregation**)
- A teacher teaches a student. (**Association**)

Key Takeaway

- ★ Association = Related
- Aggregation = Has-a (independent)
- Composition = Part-of (dependent) 😊

Mini UML Legend

- Association (simple line)
- ◇——— Aggregation (hollow diamond)



QUICK RECAP & BEST PRACTICES

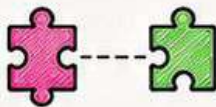
Abstraction hides details, relationships connect objects and together they help us build systems that are **easy** to **understand**, **extend** and **maintain**. 😊

1. All Relationships at a Glance

Type	How it looks (UML)	Key Point	Example
Association	A — B	General relationship. Objects are independent .	Teacher — Student Book — Author
Aggregation	A — ◇ — B	Has-a (weak). Part can exist separately .	Department ◇ Employee Team ◇ Player
Composition	A — ◆ — B	Part-of (strong). Part cannot exist without whole.	House ◆ Room Car ◆ Engine
Abstraction	A — ▷ B	Shows implementation hiding .	PaymentService —▷ UPIPaymentService

2. When to Prefer What?

- Use **Association** for general connections between independent objects.



- Use **Aggregation** when the **whole** has parts but parts can live on their own.



- Use **Composition** when the whole owns its parts and they die with the whole.



- Use **Abstraction** when you want to define



3. Best Practices

- ✓ Program to **interfaces**, not implementations.
- ✓ Prefer **composition** over inheritance.
- ✓ Keep relationships **simple** and meaningful.
- ✓ Follow **single responsibility** principle.
- ✓ Design for change. Keep it **flexible**! 😊

4. Mini Cheat Sheet

- ★ Abstraction → **What**, not **How**
- ★ Association → Connected, **independent**
- ★ Aggregation → Has-a, parts can exist **independently**
- ★ Composition → Part-of, parts **depend** on whole
- ★ Stronger Relationship → **Composition**



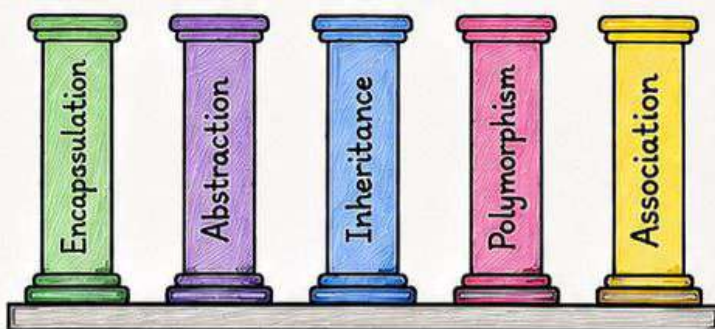
OOPs SUMMARY & CONCLUSION

OOPs is **not just** about writing code, it's about designing **smart, reusable and maintainable** systems. 😊

Key Takeaways

- ✓ Classes & Objects are the building blocks of OOPs.
- ✓ **Encapsulation** protects data and hides implementation.
- ✓ **Abstraction** shows only what is necessary, hides the rest.
- ✓ **Inheritance** promotes code reusability and establishes hierarchy.
- ✓ **Polymorphism** lets one interface behave in many ways.
- ✓ Relationships define how objects interact and depend on each other.

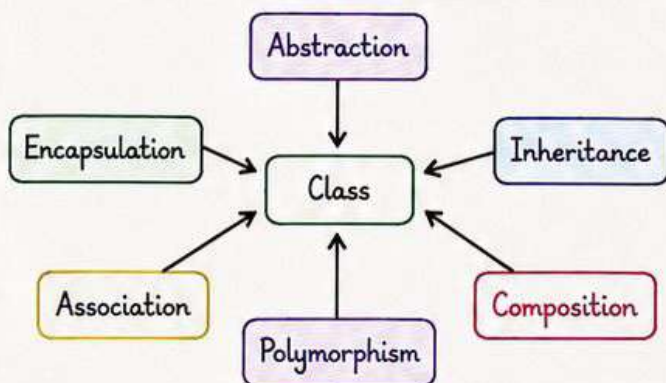
OOPs Pillars



Remember :

Good OOP design leads to **better** code, better products and happier developers! 😊

How Everything Connects



★ Together they help build systems that are **flexible, scalable** and **easy to evolve**. 😊

Real-life Analogy



Car is a class.
Toyota, Honda are **objects**.



Encapsulation is like keeping the engine details inside the car.



Abstraction is like driving the car – you don't know how the engine works.



Inheritance is like family – child gets qualities from parent.



Polymorphism is like remote – same buttons, different devices.